

# PHP-basics

José Manuel Sánchez Álvarez

## Contents

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>PHP Basics</b>                          | <b>1</b> |
| 1.1      | Setting up the environment . . . . .       | 1        |
| 1.2      | Your first PHP script . . . . .            | 2        |
| 1.3      | Variables and Data Types . . . . .         | 2        |
| 1.4      | Variable scope . . . . .                   | 3        |
| 1.5      | Expressions, operators, operands . . . . . | 4        |

## 1 PHP Basics

Welcome to the first part of my book, where I will teach you the basics of PHP, a popular and powerful scripting language for web development. PHP stands for PHP: Hypertext Preprocessor, which is a recursive acronym that reflects how PHP can process text and embed code within HTML documents. In this chapter, you will learn how to set up your environment, write your first PHP script, and use variables and data types in PHP.

### 1.1 Setting up the environment

Before you can start writing PHP code, you need to have a web server that can run PHP scripts. There are many options for web servers, such as Apache, Nginx, or IIS, but for simplicity, we will use a built-in web server that comes with PHP. To use the built-in web server, you need to have PHP installed on your computer. You can download PHP from the official website (<https://www.php.net/downloads.php>) and follow the instructions for your operating system. Once you have installed PHP, you can open a terminal or command prompt and navigate to the folder where you want to store your PHP files. Then, you can start the web server by typing the following command:

```
php -S localhost:8000
```

This will start a web server on your local machine, listening on port

1. You can access the web server by opening a browser and typing

<http://localhost:8000> in the address bar. You should see a message saying "No input file specified.", which means that the web server is working but there is no PHP file to execute.

## 1.2 Your first PHP script

To create your first PHP script, you need to create a file with the .php extension in the folder where you started the web server. For example, you can create a file called index.php and open it with a text editor. Then, you can write the following code in the file:

```
<?php
echo "Hello, world!";
?>
```

The <?php and ?> tags indicate the start and end of a PHP code block. The echo statement outputs a string to the browser. To run this script, you need to save the file and refresh the browser. You should see "Hello, world!" displayed on the web page.

## 1.3 Variables and Data Types

A variable is a container that holds a value that can change during the execution of a script. In PHP, you can declare a variable by using the \$ sign followed by a name. For example:

```
<?=  
$name = "Alice";
```

This assigns the string "Alice" to the variable \$name. You can use variables in expressions or output them using echo. For example:

```
<?=  
echo "Hello, $name!";
```

This will output "Hello, Alice!" to the browser.

PHP supports different types of data that can be stored in variables, such as integers (whole numbers), floats (decimal numbers), strings (text), booleans (true or false), arrays (collections of values), objects (instances of classes), resources (handles to external resources), and null (no value). You can check the type of a variable by using the gettype() function. For example:

```
<?=  
$age = 25;  
echo gettype($age);
```

This will output "integer" to the browser.

PHP is a *loosely typed* language, which means that you do not have to specify the type of a variable when you declare it. PHP will automatically convert the type of a variable depending on how you use it. For example:

```
<?=  
$age = 25;  
$age = "twenty-five";  
echo gettype($age);
```

This will output "string" to the browser, because \$age was assigned a string value.

## 1.4 Variable scope

The scope of a variable is the part of the script where the variable is accessible. In PHP, there are three main scopes: global, local, and static.

1. **Global scope:** A variable declared outside any function or class is in the global scope and can be accessed from anywhere in the script. However, to access a global variable from inside a function or class, you need to use the global keyword or the \$GLOBALS array. For example:

```
<?=  
$x = 10; // global variable  
  
function test() {  
    global $x; // access global variable using global keyword  
    echo $x; // output 10  
}  
  
test();  
  
function test2() {  
    echo $GLOBALS['x']; // access global variable using $GLOBALS array  
}  
  
test2();
```

2. **Local scope:** A variable declared inside a function or class is in the local scope and can only be accessed from within that function or class. For example:

```
<?=  
function test3() {  
    $y = 20; // local variable  
    echo $y; // output 20  
}  
  
test3();  
  
echo $y; // error: undefined variable
```

3. **Static scope:** A static variable is declared inside a function using the static keyword and retains its value between function calls. For example:

```
<?=  
function test4() {  
    static $z = 0; // static variable  
    $z++;  
    echo $z; // output 1, 2, 3, ... on each function call  
}  
  
test4();
```

```
test4();  
test4();
```

## 1.5 Expressions, operators, operands

An expression is a combination of values, variables, operators, and functions that evaluates to a single value. For example:

```
<?=  
$sum = 2 + 3; // expression that evaluates to 5
```

An operator is a symbol that performs a specific operation on one or more operands. An operand is a value or variable that is used by an operator. For example:

```
<?=  
2 + 3 // + is an operator, 2 and 3 are operands
```

PHP supports various types of operators, such as arithmetic, assignment, comparison, logical, string, array, and conditional operators. You can find a complete list of operators and their precedence and associativity rules in the [PHP manual](#)

Some examples of operators and their usage are:

1. Arithmetic operators: Perform basic mathematical operations such as addition (+), subtraction (-), multiplication (\*), division (/), modulus (%), and exponentiation (\*\*). For example:

```
<?=  
$a = 10;  
$b = 3;  
echo $a + $b; // output 13  
echo $a - $b; // output 7  
echo $a * $b; // output 30  
echo $a / $b; // output 3.3333333333333  
echo $a % $b; // output 1  
echo $a ** $b; // output 1000
```

2. Assignment operators: Assign a value to a variable or modify the value of a variable by another value. The basic assignment operator is (=), which assigns the right operand to the left operand. For example:

```
<?=  
$c = 5; // assign 5 to $c
```

3. There are also compound assignment operators that combine an arithmetic or string operator with the assignment operator. For example:

```
<?=  
$c += 2; // equivalent to $c = $c + 2;  
$c -= 2; // equivalent to $c = $c - 2;  
$c *= 2; // equivalent to $c = $c * 2;  
$c /= 2; // equivalent to $c = $c / 2;  
$c %= 2; // equivalent to $c = $c % 2;
```

```
$c **= 2; // equivalent to $c = $c ** 2;
$c .= "hello"; // equivalent to $c = $c . "hello";
```

4. Comparison operators: Compare two values and return a boolean value (true or false) based on the result of the comparison. There are two types of comparison operators: loose and strict. Loose comparison operators compare the values without checking their types, while strict comparison operators compare both the values and their types. For example:

```
<?=  
$d = "10";  
$e = 10;  
  
echo ($d == $e); // output true (loose equality)  
echo ($d === $e); // output false (strict equality)  
echo ($d != $e); // output false (loose inequality)  
echo ($d !== $e); // output true (strict inequality)  
echo ($d < $e); // output false (less than)  
echo ($d > $e); // output false (greater than)  
echo ($d <= $e); // output true (less than or equal to)  
echo ($d >= $e); // output true (greater than or equal to)
```

5. Logical operators: Perform logical operations on one or more boolean values and return a boolean value based on the result of the operation. There are four logical operators: and (&&), or (||), xor (^), and not (!). For example:

```
<?=  
$f = true;  
$g = false;  
  
echo ($f && $g); // output false (logical and)  
echo ($f || $g); // output true (logical or)  
echo ($f ^ $g); // output true (logical xor)  
echo (!$f); // output false (logical not)
```

6. String operators: Concatenate two strings or append one string to another. There are two string operators: concatenation (.) and concatenation assignment (.=). For example:

```
<?=  
$h = "Hello";  
$i = "world";  
  
echo ($h . " " . $i); // output Hello world (concatenation)  
$h .= " "; // append a space to $h  
$h .= $i; // append $i to $h  
echo ($h); // output Hello world (concatenation assignment)
```

7. Array operators: Combine two arrays or compare two arrays. There are five array operators: union (+), equality (==), identity (===), inequality (!=), and non-identity (!==). For example:

```

<?=
$j = array("a" => "apple", "b" => "banana");
$k = array("c" => "cherry", "d" => "durian");

print_r($j + $k); // output Array ([a] => apple [b] => banana
                  // [c] => cherry [d] => durian) (array union)
echo ($j == $k); // output false (array equality)
echo ($j === $k); // output false (array identity)
echo ($j != $k); // output true (array inequality)
echo ($j !== $k); // output true (array non-identity)

```

8. Conditional operator: Also known as the ternary operator, it evaluates a condition and returns one of two values based on whether the condition is true or false. The syntax is `condition ? value_if_true : value_if_false`. For example:

```

<?=
$l = 10;
$m = ($l > 5) ? "greater than 5" : "not greater than 5";
echo $m; // Output: "greater than 5"

```